Automated Code-centric Software Vulnerability Assessment: How Far Are We? An Empirical Study in C/C++

Anh The Nguyen; Triet Huynh Minh Le; M. Ali Babar (ESEM 2024)
Presented by: Antara Arifa Mullick
Islamic University of Technology

Problem Statement

Software vulnerabilities are weaknesses in source code that attackers can exploit to compromise systems. Because large software projects contain more vulnerabilities than can be fixed immediately, developers must prioritize remediation. Severity is commonly assessed using CVSS, but CVSS scores are typically available only after public disclosure in databases such as the NVD, reducing their usefulness during early development. Therefore, automated techniques are needed to estimate vulnerability severity directly from source code at the function level to support earlier and more effective remediation prioritization.
This study focuses on function-level assessment for C/C++ source code.

Motivation

- C/C++ underpin many critical systems and are frequently affected by severe, high-impact vulnerabilities.
- Developers need more than vulnerability detection—severity information (exploitability and impact) is required to prioritize fixes.
- CVSS/NVD severity information is often unavailable until after public disclosure, delaying early remediation decisions.
- Automated function-level vulnerability severity assessment from source code can enable earlier triage and better risk mitigation.

Research Objectives

- Investigate whether vulnerability characteristics can be predicted directly from vulnerable C/C++ functions using machine learning and deep learning models.
- Compare the performance of ML and DL approaches for function-level software vulnerability assessment.
- Evaluate multi-task learning for predicting multiple CVSS metrics simultaneously from source code.
- Predict the seven CVSSv2 metrics (Access Vector, Access Complexity, Authentication, Confidentiality, Integrity, Availability, Severity).
- Identify the most effective modeling approaches for automated vulnerability severity assessment.

Methodology

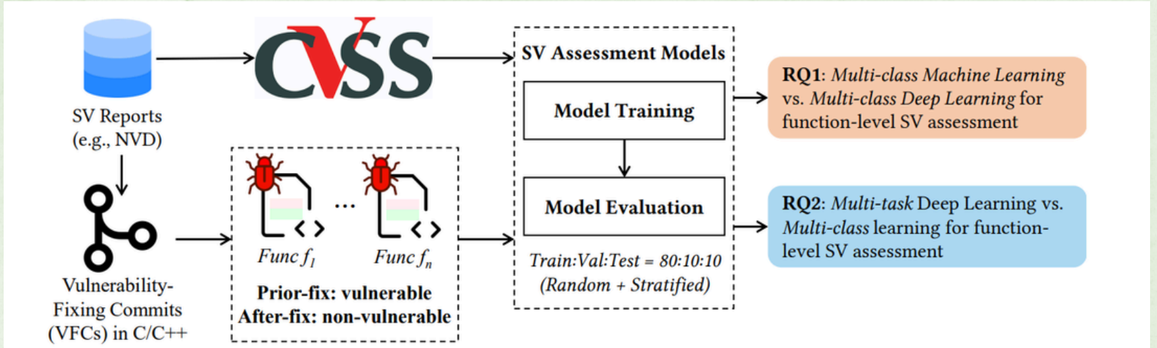


Figure 2: Methodology used to answer the research questions.

Results

Classifier	Word2Vec	fastText	BoT	BoST	Average (by classifier)
LR	0.469	0.481	0.630	0.653	0.558
SVM	0.514	0.515	0.629	0.625	0.571
KNN	0.633	0.650	0.559	0.579	0.605
RF	0.538	0.546	0.576	0.609	0.567
XGB	0.513	0.533	0.582	0.612	0.560
LGBM	0.628	0.631	0.634	0.680	0.643
Average (by feature)	0.549	0.559	0.602	0.626	

Feature

MCC: 0.70, 0.65, 0.60, 0.55, 0.50, 0.45

Figure 4: Average performance (MCC) of ML combinations (classifiers and features) for function-level SV assessment.

- LightGBM + Bag-of-Subtokens (BoST) achieved the best performance among ML models (MCC = 0.680) for function-level vulnerability assessment in C/C++.
- CodeBERT was the best-performing multi-class DL model, outperforming graph-based and sequence-based architectures.
- ML models achieved comparable performance to DL while requiring ~75% less training time.
- Multi-task learning substantially improved DL effectiveness, increasing MCC by ~8–22% over multi-class models.
- Multi-task CodeBERT achieved the best overall performance, outperforming the best ML model by ~8% MCC on average.

Experimental Results

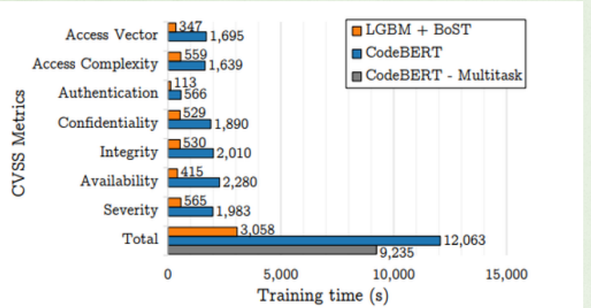


Figure 5: Training time of CodeBERT, multi-task CodeBERT and LGBM + BoST (in seconds). Note: CodeBERT multi-task does not have training time for each task.

Dataset

- Derived from Big-Vul (vulnerability-fixing commits linked to CVEs).
- ~10,000 vulnerable C/C++ functions.
- 7 CVSS v2 Base metrics used as labels.
- Exploitability metrics: Access Vector, Access Complexity, Authentication.
- Impact metrics: Confidentiality, Integrity, Availability.
- Severity metric: Overall severity level.
- Only Base metrics used, excluding Temporal and Environmental metrics.

Contributions

- First empirical study evaluating ML and DL (including multi-task learning) for function-level vulnerability assessment in C/C++.
- Empirically compares ML vs. DL for predicting CVSS-based vulnerability characteristics directly from source code.
- Shows multi-class ML delivers competitive performance with much lower training cost.
- Demonstrates multi-task DL improves effectiveness and achieves the best overall accuracy.
- Releases dataset, code, and models to support reproducibility and future research.